

Technotsav: Frontend Study Guide

How the React client works, why it is built this way, and how to explain each screen during the live demo.

Campus Event Management System · MERN stack · Frontend = `client/` folder

How to use this guide. Sections 1 to 7 explain the foundations (stack, folders, startup, routing, login, API calls, theme). Section 8 walks through every page in the order you will click it in the demo, with the code behind it. Section 10 is a demo script saying what to point out on each screen, and Section 11 lists questions an examiner is likely to ask, with short answers. Every code snippet is copied from the real project.

Contents

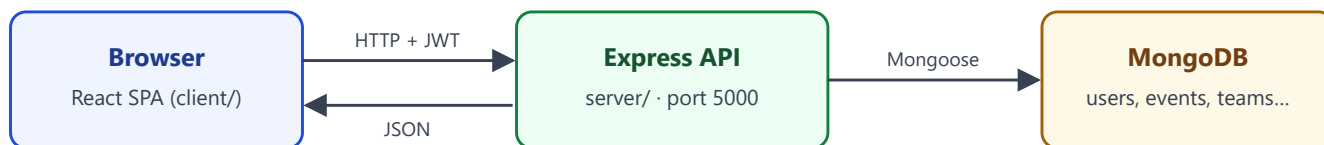
1. The big picture
2. Tech stack and why each piece was chosen
3. Folder structure
4. What happens when the app starts
5. Routing and role-based protected routes
6. Authentication: AuthContext and JWT handling
7. The Axios API layer (plus secure images and file uploads)
8. Page-by-page walkthrough (Home, Register, Login, Events, Event Detail, the three dashboards)
9. Shared components, dark mode and design choices
10. Demo walkthrough order: what to say on each screen
11. Likely examiner questions with answers

1. The big picture

Technotsav is a portal for a campus technical fest. The frontend is a **single-page application (SPA)** built with React. The browser loads one HTML page once; after that, React swaps screens in place and talks to the Express backend only for data (JSON), never for full pages.

There are three kinds of users, and the UI changes depending on who is logged in:

Role	How they get an account	What they do in the UI
Student	Signs up on the Register page	Browse events, create or join teams, register, pay by UPI QR and upload proof, see payment/attendance/results
Organizer (department rep)	Created by the Faculty Coordinator (temporary password)	Create events as drafts, submit for approval, upload payment QR, verify payments, mark attendance, enter results
Faculty Coordinator	Seeded admin account	Approve or reject events, create organizer accounts, oversee and cancel/delete any event



The frontend never talks to the database directly. Every rule is also checked on the server.

2. Tech stack and why

Technology	Version	What it does here	Why chosen
React	19	Builds the UI from components (functions that return JSX). State changes re-render only what changed.	The "R" in MERN; component reuse (one <code>StatusBadge</code> , one <code>Modal</code> used everywhere); huge ecosystem.
Vite	8	Dev server with instant hot reload, and the production build tool.	Much faster than Create React App (which is deprecated). Has a built-in proxy for <code>/api</code> .
React Router	v7 (<code>react-router-dom</code>)	Maps URLs like <code>/events/:id</code> to page components without reloading the page.	The standard router for React; supports URL params, redirects (<code><Navigate></code>) and passing state between pages.
Axios	1.x	Makes HTTP calls to the Express API.	<i>Interceptors</i> let us attach the JWT to every request in one place; automatic JSON parsing; easy file uploads with <code>FormData</code> ; can download images as blobs.
Plain CSS	one file, <code>index.css</code>	All styling, using CSS variables (design tokens).	No framework to learn or justify, small bundle, full control. CSS variables make dark mode a matter of swapping values.
lucide-react	icons	SVG icons (calendar, users, lock, sun/moon...).	Lightweight, consistent icon set, imported one icon at a time.

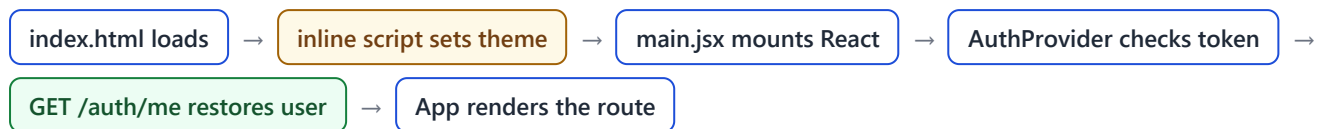
One-line answer for "why no Redux?" The only truly global state is "who is logged in", and React's built-in Context API handles that. Everything else is local to a page, so `useState` is enough. Redux would add boilerplate without benefit at this size.

3. Folder structure

```
client/
├─ index.html           ← the one HTML page; loads fonts + applies saved theme early
├─ vite.config.js       ← dev server on :5173, proxies /api → localhost:5000
├─ .env.example         ← VITE_API_URL for production deployments
└─ src/
    ├─ main.jsx         ← entry point: mounts <App/> inside Router + AuthProvider
    ├─ App.jsx          ← all routes, and which role may open each
    ├─ index.css        ← entire stylesheet: tokens, light/dark themes, components
    ├─ api/client.js     ← shared Axios instance + JWT interceptor + errorMessage()
    ├─ context/AuthContext.jsx ← logged-in user, login(), register(), logout()
    ├─ hooks/useTheme.js ← light/dark mode state
    ├─ utils/format.js   ← labels, role→dashboard map, DEPARTMENTS list, date/fee formatters
    ├─ components/
        ├─ Navbar.jsx    ← top bar, theme toggle, login/logout
        ├─ ProtectedRoute.jsx ← route guard by login + role
        ├─ ui.jsx        ← StatusBadge, Alert, Loader, EmptyState, PageHeader, Modal, Tabs
        ├─ PaymentProof.jsx ← QR display, screenshot + UTR fields, secure image loader
        ├─ EventForm.jsx ← organizer's create/edit event form (+ QR upload)
        └─ RegistrationsManager.jsx ← verify payments, attendance, results (shared)
    └─ pages/
        ├─ Home.jsx      Login.jsx Register.jsx Events.jsx EventDetail.jsx
        ├─ StudentDashboard.jsx OrganizerDashboard.jsx FacultyDashboard.jsx
        └─ NotFound.jsx
```

The idea behind the split: `pages/` are full screens tied to a URL; `components/` are reusable pieces; `api/`, `context/`, `hooks/` and `utils/` hold logic with no UI. This "separation of concerns" means, for example, that if the API address changes, only `api/client.js` changes.

4. What happens when the app starts



Step 1, theme first. A tiny script in `index.html` runs before React loads, so a dark-mode user never sees a white flash:

`client/index.html`

```
var t = localStorage.getItem("technotsav_theme");
if (t !== "light" && t !== "dark")
  t = matchMedia("(prefers-color-scheme: dark)").matches ? "dark" : "light";
document.documentElement.dataset.theme = t;
```

Step 2, providers wrap the app. The order matters: the Router is outermost so every component can navigate, and AuthProvider is inside it so every page can ask "who is logged in?".

`client/src/main.jsx`

```
createRoot(document.getElementById('root')).render(
  <StrictMode>
    <BrowserRouter>
      <AuthProvider>
        <App />
      </AuthProvider>
    </BrowserRouter>
  </StrictMode>
);
```

`StrictMode` is a development-only helper that highlights unsafe patterns; it has no effect in production.

Step 3, dev proxy. In development React runs on port 5173 and Express on 5000. Vite forwards any `/api` request to Express, so the code can use short relative URLs and there is no CORS problem:

`client/vite.config.js`

```
server: {
  port: 5173,
  proxy: { '/api': 'http://localhost:5000' },
},
```

5. Routing and role-based protected routes

All routes live in one file, `App.jsx`, so the whole site map is visible at a glance. The `Navbar` sits outside `<Routes>`, so it stays on screen while pages change.

URL	Page	Who can open it
/	Home	Everyone
/login, /register	Login, Register	Everyone (logged-in users are redirected away)
/events	Events list	Everyone
/events/:id	Event detail	Everyone (only students can register)
/dashboard	Redirect	Any logged-in user → sent to their own dashboard
/dashboard/student	StudentDashboard	Role <code>student</code>
/dashboard/organizer	OrganizerDashboard	Role <code>organizer</code>
/dashboard/faculty	FacultyDashboard	Role <code>faculty_coordinator</code>
*	NotFound	Any unknown URL

client/src/App.jsx (excerpt)

```
<Route path="/events/:id" element={<EventDetail />} />
<Route
  path="/dashboard/organizer"
  element={
    <ProtectedRoute roles={['organizer']}>
      <OrganizerDashboard />
    </ProtectedRoute>
  }
/>
<Route path="*" element={<NotFound />} />
```

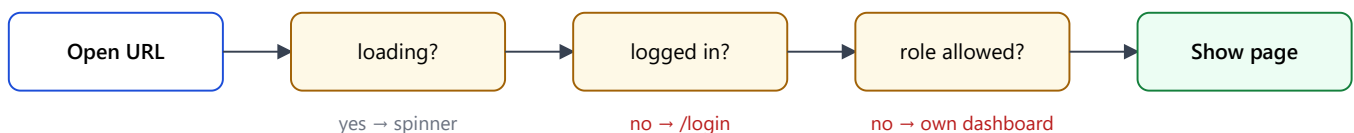
The ProtectedRoute guard

A wrapper component. It checks three things in order before showing the page:

client/src/components/ProtectedRoute.jsx

```
export default function ProtectedRoute({ roles, children }) {
  const { user, loading } = useAuth();
  const location = useLocation();

  if (loading) return <Loader />; // 1. still checking the token
  if (!user) return <Navigate to="/login" replace
    state={{ from: location.pathname }} />; // 2. not logged in
  if (roles && !roles.includes(user.role))
    return <Navigate to={DASHBOARD_PATH[user.role]} replace />; // 3. wrong role
  return children; // allowed
}
```



- **Why the loading check?** On a page refresh the user object is briefly empty while `/auth/me` runs. Without this check a logged-in user would be kicked to the login page for a split second.
- `state={{ from }}` remembers where the user was going; after login they are sent back there instead of to a generic page.

- **replace** replaces the history entry, so the browser Back button does not bounce the user into the blocked page again.

Important point to say: this guard only hides pages in the UI for a good experience. Real security is on the server: every API route checks the JWT and the role again. Someone who edits the frontend code in their browser still cannot fetch data they are not allowed to see.

The role-to-dashboard mapping is one small object, reused by the Navbar, Login and the guard:

client/src/utils/format.js

```
export const DASHBOARD_PATH = {
  student: '/dashboard/student',
  organizer: '/dashboard/organizer',
  faculty_coordinator: '/dashboard/faculty',
};
```

6. Authentication: AuthContext and JWT handling

The problem it solves: the Navbar, every dashboard and the route guard all need to know who is logged in. Passing the user down through props would be messy ("prop drilling"). React **Context** makes one value available to any component that calls `useAuth()`.

JWT in one sentence: when you log in, the server returns a signed token (JSON Web Token) that encodes your user id; the browser sends it back on every request as `Authorization: Bearer <token>`, and the server verifies the signature to know who you are without a session table.

client/src/context/AuthContext.jsx

```
export function AuthProvider({ children }) {
  const [user, setUser] = useState(null);
  const [loading, setLoading] = useState(true);

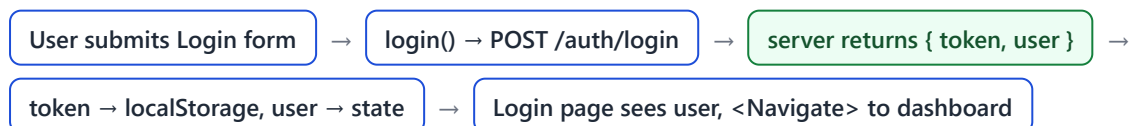
  // On first load: if a token is saved, ask the server who it belongs to.
  useEffect(() => {
    if (!localStorage.getItem(TOKEN_KEY)) { setLoading(false); return; }
    api.get('/auth/me')
      .then(({ data }) => setUser(data.user))
      .catch(() => localStorage.removeItem(TOKEN_KEY)) // expired/invalid → forget it
      .finally(() => setLoading(false));
  }, []);

  const saveSession = ({ token, user: account }) => {
    localStorage.setItem(TOKEN_KEY, token);
    setUser(account);
    return account;
  };

  const login = useCallback(async (email, password) => {
    const { data } = await api.post('/auth/login', { email, password });
    return saveSession(data);
  }, []);

  const logout = useCallback(() => {
    localStorage.removeItem(TOKEN_KEY);
    setUser(null);
  }, []);
  ...
}
export const useAuth = () => useContext(AuthContext);
```

The full login flow



- **Where is the token stored?** `localStorage` under the key `technotsav_token`. It survives page refreshes and browser restarts, which is why you stay logged in.
- **How do we stay logged in after refresh?** React state is lost on refresh, but the token is not. The `useEffect` above calls `GET /auth/me` to rebuild the user object. If the token has expired the server returns 401, and we delete the token.
- **Logout** is purely client-side: delete the token and clear the user. Without the token, the next API call is anonymous.

- `useCallback` and `useMemo` keep the functions and the context value stable between renders, so components using `useAuth()` do not re-render unnecessarily.
- **The role is never chosen by the user.** It comes from the account in the database, returned inside `user.role`. The Register page has no role selector on purpose: anyone who signs up is a student.

7. The Axios API layer

Every page imports one shared Axios instance instead of calling `fetch` directly. This is the single place that knows the API address and how to authenticate.

client/src/api/client.js

```
export const TOKEN_KEY = 'technotsav_token';

const api = axios.create({
  baseURL: import.meta.env.VITE_API_URL || '/api', // '/api' in dev (proxied), full URL in production
});

// Runs before EVERY request: attach the JWT if we have one.
api.interceptors.request.use((config) => {
  const token = localStorage.getItem(TOKEN_KEY);
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

// Turns any failure into a readable sentence for the user.
export const errorMessage = (err) =>
  err?.response?.data?.message || err?.message || 'Something went wrong. Please try again.';
```

- **Interceptor** = a function Axios runs on every request. Because of it, no page ever writes auth headers by hand; a page just calls `api.get('/registrations/me')`.
- `errorMessage()` shows the server's own message (for example "Registration deadline has passed") in an `<Alert>`, falling back to a generic sentence if the server is unreachable.
- `VITE_API_URL`: environment variable used when the frontend and backend are deployed on different domains. Vite only exposes variables that start with `VITE_` to browser code.

The standard data-loading pattern used on every page

```
const [events, setEvents] = useState(null); // null = still loading

useEffect(() => {
  api.get('/events')
    .then(({ data }) => setEvents(data.events))
    .catch((err) => setError(errorMessage(err)));
}, []); // [] = run once when the page opens

if (!events) return <Loader />; // then: empty state, or the list
```

Three visual states are always handled: **loading** (`Loader`), **empty** (`EmptyState` with a helpful message) and **data**. Errors appear in an `Alert`.

Uploading files: FormData

JSON cannot carry an image, so registrations with a payment screenshot are sent as `multipart/form-data`. A helper builds the body; the server (Multer) reads the file field `paymentScreenshot`:

client/src/components/PaymentProof.jsx

```
export const proofFormData = (fields, proof) => {
  const form = new FormData();
  Object.entries(fields).forEach(([key, val]) => val && form.append(key, val));
  if (proof?.file) form.append('paymentScreenshot', proof.file);
  if (proof?.utr) form.append('utrNumber', proof.utr);
  return form;
};
```

Showing protected images (payment QR and screenshots)

Payment screenshots are private: only the student, that event's organizer and faculty may see them. A normal `` cannot send the JWT header, so the server would refuse it. The `ApiImage` component solves this by downloading the image with Axios (which adds the token) and displaying it from a temporary **blob URL**:

```
useEffect(() => {
  let url; let cancelled = false;
  api.get(path, { responseType: 'blob' }) // token added by the interceptor
    .then(({ data }) => {
      if (cancelled) return;
      url = URL.createObjectURL(data); // "blob:http://..." local URL
      setSrc(url);
    })
    .catch(() => !cancelled && setFailed(true));
  return () => { cancelled = true; if (url) URL.revokeObjectURL(url); }; // free memory
}, [path]);
```

The cleanup function frees the memory when the component closes, and the `cancelled` flag prevents updating a component that has already been removed.

8. Page-by-page walkthrough

Pages are listed in the order you will most likely click them in the demo.

8.1 Navbar (visible on every page)

- Brand link to Home, an **Events** link, and a **Dashboard** link that only appears when logged in and points to `DASHBOARD_PATH[user.role]`.
- Right side: the **sun/moon theme toggle**, then either the user's name, role and department with a Log out button, or Log in / Sign up buttons.
- Uses `NavLink` (not `Link`), which automatically marks the current page's link as active for styling.

```
{user ? (  
  <div className="nav-identity">  
    <span className="nav-name">{user.name}</span>  
    <span className="nav-role">{ROLE_LABELS[user.role]}{user.department ? ` · ${user.department}` : ''}</span>  
  </div>  
  <Link to="/login">Log in</Link> ... )}
```

8.2 Home (/)

- A short intro with two buttons: **Browse events**, and either **Go to dashboard** (logged in) or **Log in** (guest). This is conditional rendering based on `useAuth()`.
- "Open for registration": fetches `GET /events`, keeps only published ones and shows the first five with date, team size and fee.

```
api.get('/events')  
  .then(({ data }) => setEvents(data.events.filter((e) => e.status === 'published').slice(0, 5)))
```

8.3 Register (/register)

- Student self sign-up: name, email, password (minimum 6 characters, checked in the browser with `minLength` and again on the server), college, phone.
- **No role field**: the server always creates a student. Organizers are created only by faculty.
- Uses a **controlled form**: one state object, and one `update` function for all inputs using the input's `name`:

```
const [form, setForm] = useState({ name: '', email: '', password: '', college: '', phone: '' });  
const update = (e) => setForm({ ...form, [e.target.name]: e.target.value });
```

- After successful sign-up `register()` saves the token, `user` becomes set, and the page redirects the student straight to `/events`.
- A `busy` flag disables the button and changes its text to "Creating account..." to prevent double submissions.

8.4 Login (/login)

- One login form for all three roles. The role comes from the account, so the user never picks it.
- If a user is already logged in (or has just logged in), the page renders a redirect instead of the form:

```
if (user) return <Navigate to={location.state?.from || DASHBOARD_PATH[user.role]} replace />;
```

So a student who clicked "Log in" from an event page lands back on that event; everyone else lands on their dashboard. Wrong password → the server's message appears in a red `Alert`.

8.5 Events (/events)

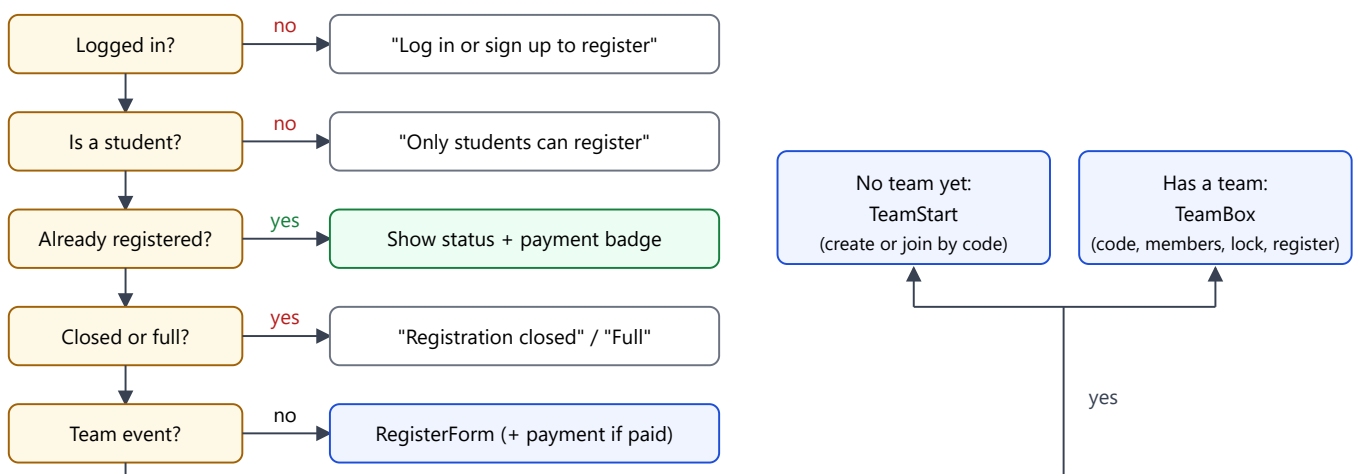
- A card grid of every event approved by faculty. Drafts and pending events never reach this page because the API filters them out for everyone.
- **Category filter:** the dropdown options come from `GET /events/categories`. Changing it re-fetches from the server (the effect depends on `[category]`).
- **Search** is done in the browser with `useMemo`, so typing feels instant and does not hit the server on every keystroke:

```
const visible = useMemo(() => {
  if (!events) return null;
  const q = query.trim().toLowerCase();
  return q ? events.filter((e) => `${e.title} ${e.department}`.toLowerCase().includes(q)) : events;
}, [events, query]);
```

- Each card computes **seats left** and whether registration is **closed** (deadline passed) or **full**, and shows it in red when so.

8.6 Event Detail (/events/:id): the most important page

Reads the id from the URL with `useParams()` and loads `GET /events/:id`. The left side shows facts (start, end, deadline, venue, format, fee), description, rules and prizes. The right side is the **RegistrationPanel**, which decides what this visitor can do:



This is a chain of early `return`s in one component, which keeps each case readable:

```
if (!user) return <p><Link to="/login" state={{ from: `/events/${event._id}` }}>Log in</Link> ...</p>;
if (!isStudent) return <p>You are viewing this event as {ROLE_LABELS[user.role]}. Only students can register.</p>;
if (registration) return ( ...registered box with payment StatusBadge... );
if (event.status !== 'published' || deadlinePassed) return <p>Registration is closed for this event.</p>;
if (seatsLeft === 0) return <p>This event is full.</p>;
if (!isTeamEvent) return <RegisterForm event={event} ... />;
return team ? <TeamBox ... /> : <TeamStart ... />;
```

To know whether the student is already registered or in a team, it loads two things **in parallel** with `Promise.all` (faster than one after the other):

```
const [regs, teams] = await Promise.all([
  api.get('/registrations/me'),
  isTeamEvent ? api.get(`/teams/event/${event._id}`) : Promise.resolve({ data: { teams: [] } }),
]);
```

Team flow

- **TeamStart**: create a team (POST /teams with a name) or join one (POST /teams/join with a 6-character invite code, auto-uppercased as you type).
- **TeamBox**: shows the invite code with a copy button (navigator.clipboard), the members with the leader marked, and "Members 2/4 (minimum 2)".
- Only the **team leader** sees Lock team and Register; other members see "Your team leader registers the team for the event." The Register button appears only when the team has at least the minimum number of members.

Payment flow: QR, screenshot and UTR

For a paid event (entryFee > 0), the register form includes PaymentProofFields:

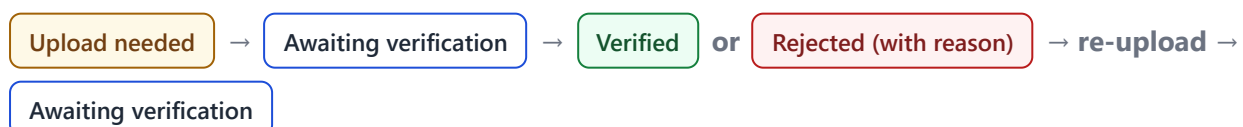
1. Step 1: the organizer's **UPI QR code** is shown (loaded securely by PaymentQr → ApiImage). The student scans it and pays the fee in any UPI app.
2. Step 2: the student attaches the **payment screenshot** (JPG, PNG or WEBP, max 2 MB) and types the **UTR number** (the 12 to 22 character transaction reference from the UPI app).
3. The Register button stays disabled until both are filled: disabled={busy || (paid && (!proof.file || !proof.utr))}.
4. Submit sends POST /registrations as FormData (Section 7). The payment status becomes *Awaiting verification*.

client/src/pages/EventDetail.jsx, RegisterForm

```
const submit = async (e) => {
  e.preventDefault();
  setBusy(true);
  try {
    const body = proofFormData({ eventId: event._id, teamId }, paid ? proof : null);
    const { data } = await api.post('/registrations', body);
    await onDone(data.message); // show success, reload registration + seat count
  } catch (err) { onError(err); }
  finally { setBusy(false); }
};
```

The whole payment life cycle, as the status badges show it:

Free events are simply *Not required*. Paid events move through:



Why manual UPI proof instead of a payment gateway? A gateway (Razorpay, etc.) needs a registered business, KYC and fees. A college fest collects small fees by UPI QR already, so the app digitises that process: the organizer checks the screenshot and UTR against their bank app and clicks Verify or Reject. The UTR is unique per transaction, so it also helps catch reused screenshots.

8.7 Student Dashboard (/dashboard/student)

- One table of all the student's registrations (GET /registrations/me) with columns **Event, Team, Payment, Attendance, Result**.
- If any payment was rejected, a red banner at the top counts them and the row shows the organizer's reason plus a **Re-upload** button, which opens `ReuploadModal` and sends `PUT /registrations/:id/payment-proof`.
- In a team registration only the person who registered (the leader) can re-upload: `r.user?._id === user._id`; others see "Your team leader needs to re-upload."
- **View proof** opens a modal with the UTR and the screenshot (via `ProofImage`).
- Result column shows the position (Winner, Runner-up...) once the organizer enters it, otherwise "Not announced". A cancelled event gets a red "Event cancelled" badge.

8.8 Organizer Dashboard (/dashboard/organizer)

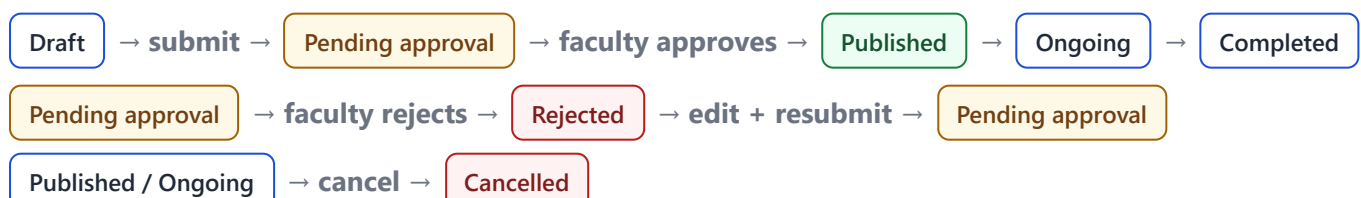
One page with three "modes", switched by a `view` state object instead of separate URLs:

```
// view: { mode: 'list' } | { mode: 'create' } | { mode: 'edit', event } | { mode: 'registrations', eventId }  
const [view, setView] = useState({ mode: 'list' });
```

- **List mode:** all of this organizer's events (GET /events?scope=mine) with filter tabs All / Drafts / Pending / Published / Rejected, each showing a count.
- Rejected events show the faculty's **rejection reason** in red under the badge.
- Which buttons appear depends on the event status, defined in one lookup table that mirrors the server's rules:

```
const CAN = {  
  edit:      ['draft', 'pending_approval', 'rejected', 'published'],  
  submit:    ['draft', 'rejected'],  
  withdraw:  ['pending_approval'],  
  registrations: ['published', 'ongoing', 'completed', 'cancelled'],  
  cancel:    ['published', 'ongoing'],  
  delete:    ['draft', 'pending_approval', 'rejected', 'cancelled'],  
};  
...  
{CAN.submit.includes(e.status) && (  
  <button onClick={() => run(() => api.put(`/events/${e._id}/submit`))}>Submit for approval</button>  
)}
```

Event life cycle (what those buttons move between):



- Destructive actions (Cancel, Delete) ask first with `window.confirm`.
- **EventForm** (create/edit): title, description, category (with suggestions), venue, start/end/deadline (`datetime-local` inputs), min/max team size, capacity, entry fee, rules (one per line, turned into an array), and prizes (add/remove rows). New events are saved as **drafts**.
- If the fee is above zero, a **payment QR upload** is required, with a live local preview using `URL.createObjectURL(qrFile)`. The QR is uploaded in a second request after the event is saved, because it

needs the event's id:

```
const { data } = editing ? await api.put(`/events/${event._id}`, payload) : await api.post('/events', payload);
if (paid && qrFile) {
  const body = new FormData();
  body.append('paymentQr', qrFile);
  await api.put(`/events/${data.event._id}/payment-qr`, body);
}
```

- Editing a **published** event shows a warning: saving sends it back to faculty for re-approval, so students never see unapproved changes.
- Dates are converted to ISO strings (UTC) before sending, and back to local time for the form with the `toLocalInput` helper.

RegistrationsManager (shared by organizer and faculty)

- Opened with the **Registrations** button. Shows stat tiles (Registrations x / capacity, Awaiting verification, Payments verified, Present) and one row per registration with participant, team, payment, attendance and result.
- **Review** opens the screenshot and UTR in a wide modal with **Verify** and **Reject** (reject requires remarks, which the student sees).
- **Attendance**: mark Present/Absent (`PUT /registrations/:id/attendance`).
- **Result**: a text box with suggestions from a `<datalist>` (Winner, Runner-up, Second runner-up, Participant) saved with `PUT /registrations/:id/result` .
- Every action goes through one `act()` helper: call the API, show the message, reload the table.

8.9 Faculty Coordinator Dashboard (/dashboard/faculty)

Three tabs built with the shared `Tabs` component; the Review tab shows a count of pending events.

- **Review events:** a card for each event with status `pending_approval`: description, facts, organizer's name and email, and for paid events the **payment QR students will pay to** (so faculty can check it is legitimate). Buttons: **Approve** (`PUT /events/:id/approve`), **Reject** (opens a modal that requires a reason, `PUT /events/:id/reject`), and Full details.
- **Organizers:** a form to create organizer accounts, and the list of existing organizers. The server generates a **temporary password**, shown once with a "Copy credentials" button; faculty passes it to the student representative.
- **All events:** oversight table of every event from every department, filterable by status and department, with Registrations, Cancel and Delete (delete also removes teams and registrations, so it asks for confirmation).

The strict department dropdown

The organizer's department is chosen from a fixed list, never typed:

```
<select name="department" required value={form.department} onChange={update}>
  <option value="" disabled>Select a department</option>
  {DEPARTMENTS.map((d) => <option key={d} value={d}>{d}</option>)}
</select>
```

client/src/utils/format.js

```
// Must match DEPARTMENTS in server/utils/constants.js (the server rejects anything else).
export const DEPARTMENTS = [
  'Computer Science and Engineering', 'CSE (AIML)', 'Electronics and Communication Engineering',
  'CSE (Data Science)', 'Mechanical Engineering', 'Civil Engineering', 'Chemical Engineering',
];
```

Why strict? An organizer can only manage events of their own department. That check compares department names as text. If faculty could type freely, "CSE" and "Computer Science" would be treated as two different departments and permissions would break. A dropdown, validated again on the server, guarantees exact matches.

9. Shared components, dark mode and design choices

Reusable UI building blocks (components/ui.jsx)

Component	Purpose
StatusBadge	Coloured pill for any status. Two lookup maps turn pending_approval into the label "Pending approval" and the colour "warning".
Alert	Error/success/info message with an icon and optional close button. Renders nothing when empty, so pages can always include it.
Loader , EmptyState	Consistent loading and "nothing here yet" screens.
PageHeader	Title, subtitle and action buttons at the top of each page.
Modal	Dialog that closes on Escape or on clicking the dark backdrop; uses role="dialog" and aria-modal for screen readers.
Tabs	Tab switcher with optional icon and count, used by both dashboards.

```
export function StatusBadge({ value, children }) {  
  return <span className={`badge badge-${tone(value)}`}>{children || label(value)}</span>;  
}  
// tone('verified') → 'success' (green), tone('rejected') → 'danger' (red)
```

Dark mode

1. All colours are **CSS variables** in :root (for example --bg, --text, --primary). Components never use raw colours.
2. A second block, :root[data-theme="dark"], redefines the same variables with dark values.
3. The useTheme hook stores 'light' or 'dark', writes it to <html data-theme="..."> and saves it in localStorage. The Navbar button calls toggleTheme.
4. First visit with no saved choice follows the operating system setting (prefers-color-scheme).

```
:root { --bg: #f7f7f8; --surface: #ffffff; --text: #1f2937; --primary: #1d4ed8; }  
:root[data-theme="dark"] { --bg: #111317; --surface: #181b20; --text: #e5e7eb; --primary: #4f7cf5; }  
  
// useTheme.js  
useEffect(() => {  
  document.documentElement.dataset.theme = theme;  
  localStorage.setItem(THEME_KEY, theme);  
}, [theme]);  
const toggleTheme = () => setTheme((t) => (t === 'dark' ? 'light' : 'dark'));
```

Because every component reads variables, switching theme needs zero changes in any component: the browser simply recalculates the colours.

Design choices worth mentioning

- **Clean, minimal, dashboard style:** one brand blue, neutral greys, and muted status colours (green verified, amber pending, red rejected) so status is readable at a glance.
- **Fonts:** IBM Plex Sans for headings and Inter for body text, loaded from Google Fonts.
- **Responsive:** CSS media queries at 900px and 640px stack layouts for tablets and phones; wide tables scroll inside a .table-wrap instead of breaking the page.

- **Feedback everywhere:** buttons show "Submitting..." and disable while working; every action ends in a green or red Alert with the server's message.
- **Accessibility:** real `<label>`s on inputs, `aria-label` on icon-only buttons, `role="alert"` for errors, Escape closes modals.
- **Single source of truth:** labels, colours, role paths and departments live in `utils/format.js`, so a change is made once.
- **UI mirrors server rules** (for example the `CAN` table) to hide buttons that would fail, but the server remains the authority.

10. Demo walkthrough order

Suggested order for the live demo, with what to click and what to explain at each screen. Log in with each role in turn; use a second browser or private window if you want two roles open at once.

#	Screen / action	What to explain
1	Home page as a guest	It is a React SPA built with Vite; the navbar stays while pages change (React Router). Point out the dark-mode toggle and click it: CSS variables, remembered in localStorage.
2	Events page: search and category filter	Only faculty-approved events appear. Search is instant because it filters in the browser with <code>useMemo</code> ; category filter re-fetches from the API. Cards show seats left and deadline.
3	Try opening <code>/dashboard/faculty</code> as a guest	ProtectedRoute redirects to Login and remembers where you were going. Mention the server enforces the same rule.
4	Log in as Faculty → Organizers tab	One login for all roles; role comes from the account. Create an organizer: strict department dropdown (why), temporary password shown once.
5	Log in as Organizer → New event	EventForm, saved as a draft. For a paid event the QR upload is required, with instant preview. Show the status tabs and how buttons change with status (the <code>CAN</code> table). Click Submit for approval.
6	Faculty → Review events	Pending event card with the payment QR. Show Reject with a reason (organizer sees it), or Approve → it becomes Published and appears on the Events page.
7	Sign up as a Student	No role selector on purpose. After sign-up you are logged in immediately (token saved) and sent to Events.
8	Event detail → team event	RegistrationPanel decides what to show. Create a team, show the invite code and copy button; explain leader-only registration and minimum team size.
9	Register for a paid event	QR step then screenshot + UTR; button disabled until both are filled; sent as FormData. Status becomes Awaiting verification.
10	Student dashboard	Table of registrations with payment, attendance and result badges; View proof loads the private image securely (blob URL).
11	Organizer → Registrations	Stat tiles; Review the screenshot, Verify or Reject with remarks; mark attendance; enter a result.
12	Back to Student dashboard	Shows Verified / Present / Winner. If rejected: red banner, reason, and the Re-upload flow.
13	Refresh the page while logged in	You stay logged in: token in localStorage, <code>GET /auth/me</code> rebuilds the user. Close with a recap of the stack.

If you have only one minute of explaining left: "The frontend is a React 19 single-page app built with Vite. React Router maps URLs to pages and a ProtectedRoute component guards dashboards by role. Login state lives in a React Context; the JWT is stored in localStorage and an Axios interceptor attaches it to every request. Each role gets its own dashboard, and paid events use a UPI QR plus screenshot and UTR upload that organizers verify. Styling is plain CSS with variables, which also powers dark mode."

11. Likely examiner questions

Q1. Why React and not plain HTML/JS?

The UI changes a lot based on state (who is logged in, event status, payment status). React re-renders automatically when state changes and lets us reuse components like `StatusBadge` and `Modal` across all pages, instead of manually updating the DOM.

Q2. What is a single-page application?

The browser loads one HTML page and one JavaScript bundle. Navigation is handled by React Router in the browser, so moving between pages does not reload; only data is fetched from the API as JSON.

Q3. How does role-based access work on the frontend?

Dashboard routes are wrapped in `ProtectedRoute` with an allowed `roles` list. It shows a loader while the session is restored, redirects guests to `/login`, and redirects a wrong role to their own dashboard. The server checks the JWT and role on every API call too, so the frontend check is for user experience, not security.

Q4. Where is the JWT stored and how is it sent?

In `localStorage` under `technotsav_token`. An Axios request interceptor reads it and adds `Authorization: Bearer <token>` to every request.

Q5. Is `localStorage` safe for tokens?

It is simple and common for student and internal projects, but readable by JavaScript, so an XSS bug could leak it. React escapes rendered text by default, which reduces XSS risk. A production upgrade would be an `httpOnly` cookie plus short token expiry.

Q6. How does the user stay logged in after refreshing?

On startup `AuthProvider` finds the saved token and calls `GET /auth/me` to get the user again. If the token is invalid or expired, it is removed and the user is treated as a guest.

Q7. What is the Context API and why not Redux?

Context shares a value with every component without passing props through each level. We only have one global value (the logged-in user and login/logout functions), so Context is enough; Redux would add boilerplate for no gain.

Q8. What hooks did you use?

`useState` (local state), `useEffect` (loading data, applying the theme), `useContext` via `useAuth`, `useCallback` / `useMemo` (stable functions, fast search filtering), React Router's `useParams`, `useNavigate`, `useLocation`, plus our custom hook `useTheme`.

Q9. Why Axios instead of fetch?

Interceptors (JWT added in one place), automatic JSON parsing, errors thrown for 4xx/5xx responses, a `baseUrl`, and easy blob downloads and `FormData` uploads.

Q10. How are file uploads sent?

With a `FormData` object (multipart/form-data): fields `paymentScreenshot` and `utrNumber` for registrations, and `paymentQr` for the organizer's QR. The server's Multer middleware checks type and the 2 MB size limit.

Q11. Why can't you just use `<img src>` for the payment screenshot?

An `img` tag cannot send the Authorization header, and screenshots are private. We fetch the image with Axios as a blob and show it through `URL.createObjectURL`, then revoke the URL when done to free memory.

Q12. How does the payment verification work without a payment gateway?

The organizer uploads their UPI QR. Students pay, then upload the screenshot and UTR. The organizer compares them with their bank records and clicks Verify or Reject with a reason; a rejected student can re-upload. This avoids gateway fees and business KYC, which a college fest usually does not have.

Q13. How is dark mode implemented?

Colours are CSS variables; a `[data-theme="dark"]` block overrides them. The `useTheme` hook toggles that attribute on `<html>` and saves the choice. A small script in `index.html` applies the theme before React loads to avoid a white flash.

Q14. Why plain CSS instead of Bootstrap or Tailwind?

Full control over a small, consistent design, no extra dependency, a smaller bundle, and CSS variables give us theming for free. It also shows we understand CSS rather than relying on a framework.

Q15. Why is department a dropdown?

Permissions compare department names exactly. Free text would allow spelling variants that silently break those checks, so the list is fixed and identical on client and server.

Q16. How do you handle loading and errors?

State starts as `null` to mean loading and shows a Loader; an empty array shows an EmptyState; errors go through `errorMessage()` into an Alert with the server's message. Buttons disable while a request is running to prevent double submissions.

Q17. What happens if a student opens an event after the deadline or when it is full?

The RegistrationPanel shows "Registration is closed" or "This event is full" instead of the form. The server also rejects late or over-capacity registrations.

Q18. How do team registrations work?

A student creates a team and gets an invite code; teammates join with the code. Only the leader can lock the team and register it, and only once the minimum team size is reached. Everyone in the team then sees the registration on their dashboard.

Q19. What is the purpose of the Vite proxy?

In development React (5173) and Express (5000) run on different ports. The proxy forwards `/api` calls to Express, so the code uses relative URLs and avoids CORS errors. In production `VITE_API_URL` sets the real API address.

Q20. What would you improve next?

httpOnly cookie auth, email notifications on approval and payment status, pagination for large event lists, automated tests (React Testing Library), and optionally a real payment gateway.

Quick glossary

Component	A function returning JSX (HTML-like UI).	Props	Inputs passed to a component.
State	Data that, when changed, re-renders the UI.	Hook	A <code>use...</code> function giving components state or side effects.
JWT	Signed login token sent with each request.	Interceptor	Axios function run on every request.
Controlled input	Input whose value lives in React state.	UTR	Unique Transaction Reference of a UPI payment.